

tinyGPT: A From-Scratch C++ Implementation of a Character-Level Transformer with Hand-Derived Backpropagation

Shivansh Pratap Singh

Department of Computer Science and Engineering

G. L. Bajaj Institute of Technology and Management

Plot No. 2, Knowledge Park III, Greater Noida, Uttar Pradesh 201306, India

Email: cse23308@glbittm.ac.in

Abstract—We present tinyGPT, a character-level transformer language model implemented entirely from scratch in C++ with no machine learning or linear algebra libraries. The forward pass, backward pass, and optimizer are hand-derived on top of a small self-built matrix autograd engine. We describe two implementations: a minimal single-head, single-block model (tinygpt) and an extended version (tinygpt2) with multi-head attention, two stacked transformer blocks, and Pre-LayerNorm. Both are trained on a character-level corpus using the Adam optimizer with causal masking. We report training loss curves and generated samples for both models trained on *Alice’s Adventures in Wonderland*, showing that the architectural extensions in tinygpt2 reduce final training loss from 1.77 to 1.59 at comparable iteration counts, and to 1.40 with extended training. We discuss implementation details including a reference-cycle memory bug in the autograd engine, learning rate sensitivity, and the empirical decision to omit LayerNorm in the single-block model. The project’s goal is pedagogical: to understand attention and backpropagation through direct implementation rather than to produce a competitive language model.

Index Terms—transformer, attention mechanism, autograd, backpropagation, character-level language model, C++

I. INTRODUCTION

Transformer-based language models are almost universally built on top of established deep learning frameworks such as PyTorch or TensorFlow, which abstract away the mechanics of automatic differentiation, tensor operations, and gradient computation. This abstraction is useful for building large-scale systems but obscures the underlying mathematics for anyone trying to understand *how* a transformer actually computes its forward and backward passes.

tinyGPT is a character-level transformer written entirely in C++ using only the standard library. There is no autograd framework, no BLAS or linear algebra library, and no use of any external machine learning tooling. Every tensor operation, its corresponding backward rule, and the optimizer update are implemented by hand. The project is deliberately small in scale; it is not intended to compete with production language models but to serve as a complete, inspectable reference implementation of the components that make a transformer

work: token and positional embeddings, causal self-attention, feedforward layers, LayerNorm, cross-entropy loss, and Adam optimization.

Two versions of the model are presented. tinygpt is the minimal version: a single attention head operating over the full embedding dimension, one transformer block, and no LayerNorm. tinygpt2 extends this into a more standard architecture: four attention heads, two stacked transformer blocks, and Pre-LayerNorm applied before each sub-layer. Both models share the same underlying autograd engine and are trained identically, which allows for a controlled comparison of how these architectural choices affect training loss and generated text quality.

II. RELATED WORK

The transformer architecture was introduced for sequence modeling [1] and forms the basis of modern language models such as the GPT family [2], which stack many transformer blocks with multi-head self-attention and LayerNorm to achieve high capacity language modeling. Educational reimplementations of GPT-style models exist in Python using minimal frameworks, typically still relying on PyTorch’s autograd for gradient computation [3]. tinyGPT differs from these in that it does not use any autograd framework at all; the computation graph, topological sort, and backward pass are all implemented manually in C++, and the only tensor abstraction is a simple 2D matrix with a forward value buffer and a gradient buffer.

III. AUTOGRAD ENGINE

The core of tinyGPT is a minimal reverse-mode automatic differentiation engine built around a single `Tensor` struct:

```
struct Tensor {  
    int r, c;  
    vector<double> d, g;  
    vector<TPtr> prev;  
    function<void()> bw;  
};
```

Each tensor stores its forward values (`d`), accumulated gradients (`g`), a list of parent tensors (`prev`), and a backward

Corresponding author: Shivansh Pratap Singh (email: cse23308@glbittm.ac.in).

closure (`bw`) that routes gradients to its parents. Every operation constructs a new output tensor, records its parents, and attaches a lambda that implements the operation’s backward rule. Calling `backward()` on a loss tensor performs a topological sort over the graph reachable from the loss, initializes the loss gradient to 1.0, and walks the sorted order in reverse, invoking each tensor’s backward closure.

A notable implementation issue arose from how backward closures captured their own output tensor. Capturing a `shared_ptr` to the tensor itself inside its own backward lambda creates a reference cycle: the tensor’s `bw` closure holds a shared pointer to the tensor, and the tensor holds the closure, so neither is ever freed. This caused memory to grow unbounded during training until the process was killed by the operating system. The fix was to capture a raw pointer (`Tensor* op = o.get()`) instead of the shared pointer, breaking the cycle while relying on the caller holding the owning `shared_ptr` to keep the tensor alive for the duration it is needed.

Table I lists the tensor operations implemented, along with their backward rules.

TABLE I
AUTograd OPERATIONS

Operation	Backward Rule
<code>matmul(A, B)</code>	$dA += dO B^\top, dB += A^\top dO$
<code>add(a, b)</code>	$da += dO, db += dO$
<code>add_bias(a, b)</code>	$db = \text{column-wise sum of } dO$
<code>relu(a)</code>	$da += dO \cdot \mathbb{I}[a > 0]$
<code>scale(a, s)</code>	$da += dO \cdot s$
<code>transpose(a)</code>	$da[i, j] += dO[j, i]$
<code>causal_mask(a)</code>	pass-through on lower triangle
<code>softmax_rows(a)</code>	Jacobian via dot-product trick
<code>embed(E, idx)</code>	scatter-add into embedding rows
<code>concat_cols(ts)</code>	slice gradient back per input
<code>layernorm_rows</code>	standard LN backward formula

IV. MODEL ARCHITECTURE

A. *tinygpt*

The baseline model processes a sequence of T character indices through a single attention head and a single transformer block:

$$x = \text{Embed}(E, \text{idx}) + P[:, T] \quad (1)$$

$$Q, K, V = xW_q, xW_k, xW_v \quad (2)$$

$$\text{scores} = \frac{QK^\top}{\sqrt{d}} \quad (3)$$

$$\text{attn} = \text{softmax}(\text{causal_mask}(\text{scores})) \quad (4)$$

$$x = x + (\text{attn} \cdot V)W_o \quad (5)$$

$$x = x + \text{ReLU}(xW_1 + b_1)W_2 + b_2 \quad (6)$$

$$\text{logits} = xW_{\text{out}} + b_{\text{out}} \quad (7)$$

with $d = 64$, feedforward hidden size $d_{ff} = 256$, and context length (block) 64. No LayerNorm is used. The choice to omit LayerNorm was empirical rather than an oversight:

it was implemented, its backward pass verified against a numerical gradient check, but it converged more slowly than the version without it at this scale. LayerNorm’s benefit is primarily in stabilizing gradients across many stacked blocks; with a single block and a small dataset it adds optimization friction without a corresponding benefit.

B. *tinygpt2*

tinygpt2 extends the baseline with multi-head attention, two stacked transformer blocks, and Pre-LayerNorm applied before each sub-layer [4]:

$$x = x + \text{MHA}(\text{LN}(x)), \quad x = x + \text{FFN}(\text{LN}(x)) \quad (8)$$

Each of the $n_h = 4$ heads operates in $d_{\text{head}} = d/n_h = 16$ dimensions:

$$Q_h, K_h, V_h = xW_{q_h}, xW_{k_h}, xW_{v_h} \quad (9)$$

$$\text{attn}_h = \text{softmax} \left(\text{causal_mask} \left(\frac{Q_h K_h^\top}{\sqrt{d_{\text{head}}}} \right) \right) \quad (10)$$

$$\text{head}_h = \text{attn}_h V_h \quad (11)$$

The per-head outputs are joined column-wise and projected back to the model dimension:

$$\text{out} = \text{concat_cols}(\text{head}_0, \dots, \text{head}_{n_h-1}) W_o \quad (12)$$

This requires a new autograd primitive, `concat_cols`, whose backward pass slices the incoming gradient back into each head’s gradient buffer according to its column offset.

Pre-LN is implemented as a second new primitive, `layernorm_rows`, normalizing each row independently:

$$y_{i,j} = \gamma_j \cdot \frac{x_{i,j} - \mu_i}{\sigma_i} + \beta_j \quad (13)$$

The backward pass with respect to the input is

$$\frac{\partial L}{\partial x_{i,j}} = \frac{1}{\sigma_i} \left(\widehat{dx}_{i,j} - \frac{1}{d} \sum_k \widehat{dx}_{i,k} - \hat{x}_{i,j} \cdot \frac{1}{d} \sum_k \widehat{dx}_{i,k} \hat{x}_{i,k} \right) \quad (14)$$

where $\widehat{dx}_{i,j} = \frac{\partial L}{\partial y_{i,j}} \gamma_j$ and $\hat{x}_{i,j}$ is the normalized pre-scale value cached during the forward pass. The gradients with respect to γ and β are $\sum_i \frac{\partial L}{\partial y_{i,j}} \hat{x}_{i,j}$ and $\sum_i \frac{\partial L}{\partial y_{i,j}}$ respectively.

Table II summarizes the architectural differences between the two models.

TABLE II
ARCHITECTURE COMPARISON

Feature	<i>tinygpt</i>	<i>tinygpt2</i>
Attention heads	1 ($d = 64$)	4 ($d_{\text{head}} = 16$)
Transformer blocks	1	2
LayerNorm	none	Pre-LN
Parameter tensors	12	48
Iterations	8000	10000
Learning rate	0.0025	0.001

V. TRAINING

Both models are trained on character-level tokenization, where the vocabulary is simply the set of unique characters in the training corpus. Training minimizes cross-entropy loss, fused with softmax for numerical stability:

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^T \log p_{t,y_t}, \quad p_{t,j} = \text{softmax}(\text{logits}_t)_j \quad (15)$$

with backward rule $\frac{\partial \mathcal{L}}{\partial \text{logit}_{t,j}} = \frac{1}{T} (p_{t,j} - \mathbb{I}[j = y_t])$.

Parameters are updated with Adam [5] using standard bias-corrected moment estimates ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$). Each iteration samples a random contiguous window of length `block` from the corpus, computes the forward pass and loss, backpropagates, and applies an Adam step. Learning rate proved to be the most sensitive hyperparameter: a rate of 0.005 produced a visible loss spike mid-training at larger model sizes, while dropping to 0.0025 (tinygpt) or 0.001 (tinygpt2) eliminated the spike with no other changes.

At generation time, both models sample autoregressively at temperature 0.8, which sharpens the softmax distribution over the vocabulary relative to unscaled sampling.

VI. RESULTS

Both models were trained on *Alice’s Adventures in Wonderland* (approximately 50 KB, 64 unique characters) with a fixed random seed of 42. Table III reports training loss at selected iterations for tinygpt (8k iterations) and tinygpt2 (10k iterations), and Fig. 1 shows the corresponding loss curves.

TABLE III
TRAINING LOSS COMPARISON (8K VS 10K)

Iter	tinygpt (8k)	tinygpt2 (10k)
1000	2.44	2.43
3000	2.20	2.14
5000	1.99	1.92
7000	1.83	1.68
8000	1.77	1.64
10000	—	1.59

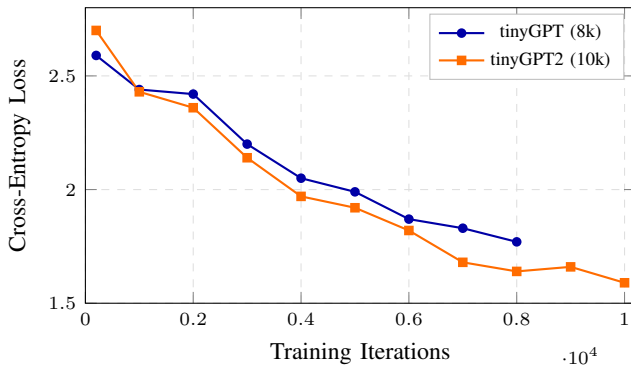


Fig. 1. Cross-entropy training loss versus training iterations for tinygpt (8k iterations, final loss 1.77) and tinygpt2 (10k iterations, final loss 1.59).

tinygpt reaches a final loss of 1.77 at 8000 iterations. tinygpt2 reaches 1.59 at 10000 iterations, already below tinygpt’s loss at a comparable point in training, as summarized in Table IV.

Continuing tinygpt2’s training run to 20000 iterations brings the loss down further, to 1.40 at 20000 iterations, with a best checkpoint of 1.342 reached at iteration 18,800 before the loss ticks back up slightly by iteration 20000. Table IV and Fig. 2 report this extended run alongside the original 8k/10k results.

TABLE IV
TRAINING LOSS COMPARISON SUMMARY

Iter	tinygpt (8k)	tinygpt2 (10k)	tinygpt2 ext. (20k)
1000	2.44	2.43	2.43
3000	2.20	2.14	2.06
5000	1.99	1.92	1.89
7000	1.83	1.68	1.68
8000	1.77	1.64	1.64
10000	—	1.59	1.59
12000	—	—	1.56
14000	—	—	1.51
16000	—	—	1.42
18000	—	—	1.39
18800	—	—	1.34 (best)
20000	—	—	1.40

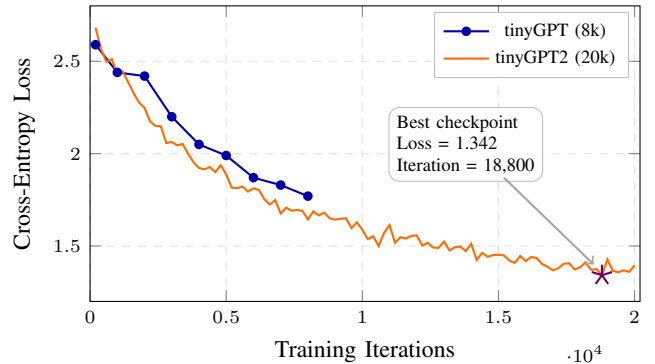


Fig. 2. Extended training loss curve for tinygpt2 out to 20000 iterations, with the best checkpoint (loss = 1.342, iteration = 18,800) marked; tinygpt (8k) shown for reference.

Generated samples reflect this gap qualitatively. At loss 1.77, tinygpt produces mostly short fragments with scattered punctuation. At loss 1.59, tinygpt2 produces longer correctly spelled words (“little”, “crown”, “frown”, “voices”, “found”) and short multi-word phrases (“a little crown a frown”, “Alice and her”). At loss 1.40 (20000 iterations), tinygpt2 produces recognizable multi-word structure and more natural punctuation placement, though the output remains far from grammatically coherent, consistent with the model’s small scale (a two-block, 64-dimensional architecture on a small corpus).

Table V shows the parameter tensor count growth from tinygpt to tinygpt2, driven by the four independent Q/K/V projections per head across two blocks plus the added LayerNorm scale/shift parameters.

TABLE V
PARAMETER TENSOR COUNT

Model	Parameter Tensors	Runtime (approx.)
tinygpt	12	2 min (8k iters)
tinygpt2	48	4 min (10k iters)
tinygpt2 (20k)	48	8 min (20k iters)

VII. DISCUSSION

The loss reduction from tinygpt to tinygpt2 (1.77 to 1.59 at comparable training budgets) is consistent with the expected effect of multi-head attention and Pre-LN: splitting the representation into multiple heads lets the model attend to different positional patterns in parallel, while Pre-LN stabilizes gradient flow through the two stacked residual paths. The fact that tinygpt2’s loss curve had not plateaued at 10000 iterations, and continued improving meaningfully to a best checkpoint loss of 1.342 at iteration 18,800 (1.40 by iteration 20000), suggests the deeper architecture has more headroom to exploit than the single-block model, which is expected since additional depth increases model capacity relative to a fixed, small corpus.

The decision to omit LayerNorm in tinygpt is a useful negative result: LayerNorm did not fail to help because of an implementation bug — it was verified correct via numerical gradient checking — but because its main benefit (stabilizing gradients across many stacked normalization boundaries) does not manifest with only one block. This matches the standard understanding of why deep transformer stacks rely on normalization while very shallow ones can do without it.

VIII. CONCLUSION

tinyGPT demonstrates that a character-level transformer, including its full forward pass, hand-derived backward pass, and Adam optimizer, can be implemented from scratch in C++ without any machine learning or linear algebra dependencies. The comparison between a minimal single-head, single-block model and an extended multi-head, two-block, Pre-LN model shows a clear and expected improvement in training loss and generated text quality, validating that the hand-implemented multi-head attention and LayerNorm backward passes are correct and functioning as intended. Natural extensions include increasing model width ($d = 128$), stacking additional blocks (3–4), and training on larger corpora, all of which the modular `Block`-based design in tinygpt2 supports without changes to the forward pass logic.

ACKNOWLEDGMENT

The author used Claude (Anthropic) [6] to assist with reformatting and language editing of the manuscript text to improve clarity and ensure adherence to IEEE formatting guidelines. AI assistance was also used during development of the underlying tinyGPT/tinygpt2 C++ implementation for code cleanup, debugging, and identification of memory usage issues; the model design, hand-derived backpropagation rules, and all reported experiments and results are the author’s own work.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI Technical Report*, 2019.
- [3] A. Karpathy, “nanoGPT,” GitHub repository, 2022. [Online]. Available: <https://github.com/karpathy/nanoGPT>
- [4] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [6] Anthropic, “Claude,” 2026. [Online]. Available: <https://www.anthropic.com/claude>